Diagramas de secuencia

Urban Mobility Hub

Plataforma multimodal de micromovilidad, logística y venta de vehículos urbanos

Ingeniería de Software II · Universidad Autónoma de Bucaramanga (UNAB)

Cómo leer este documento

Los diagramas se elaboraron a partir del código del repositorio. Hoy la interfaz habla únicamente con la interfaz DataSource (src/lib/data/datasource.ts) y la implementación activa es el adaptador mock (src/lib/data/mock.ts), con datos en memoria y localStorage. El adaptador api aún no está enchufado.

Diagramas	Qué describen
1 a 8 y 10	Comportamiento implementado hoy (adaptador mock).
9	Arquitectura objetivo definida en docs/api/openapi.yaml y src/server/domain/*, aún no conectada a la UI.

Participantes comunes

UI: componentes React. RQ: TanStack Query (caché compartida). DS: DataSource. LS: localStorage del navegador.

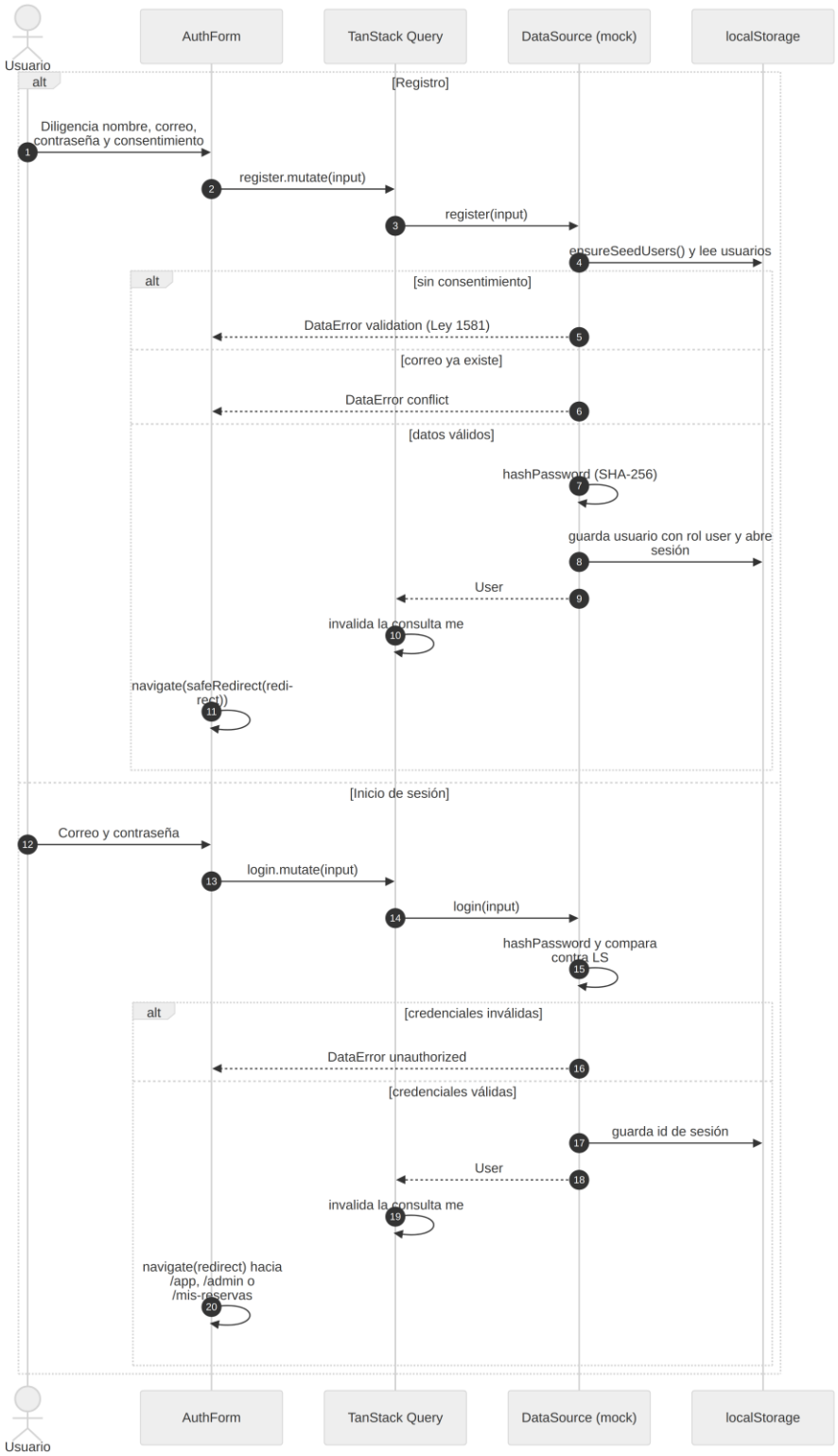
Contenido

1. Registro e inicio de sesión 2
2. Mapa en vivo: carga inicial y suscripción a la flota 3
3. Reserva de alquiler en hub o a domicilio 4
4. Compra de un vehículo del catálogo 5
5. Desbloqueo: QR, casco e inicio de viaje 6
6. Desbloqueo alterno por reconocimiento facial y registro del rostro 7
7. Gestión de reservas y fin del viaje con cálculo de costo 8
8. Simulador de alertas IoT y resolución por el operador 9
9. Telemetría IoT, trigger de seguridad y alerta en tiempo real (arquitectura objetivo) 10
10. Gestión de catálogo y hubs por el operador 11
Observaciones de coherencia detectadas en el código 12

1. Registro e inicio de sesión

Requisitos: RNF-03/05, Ley 1581 | **Fuentes:** AuthForm.tsx, queries.ts (useAuthActions), mock.ts (register, login)

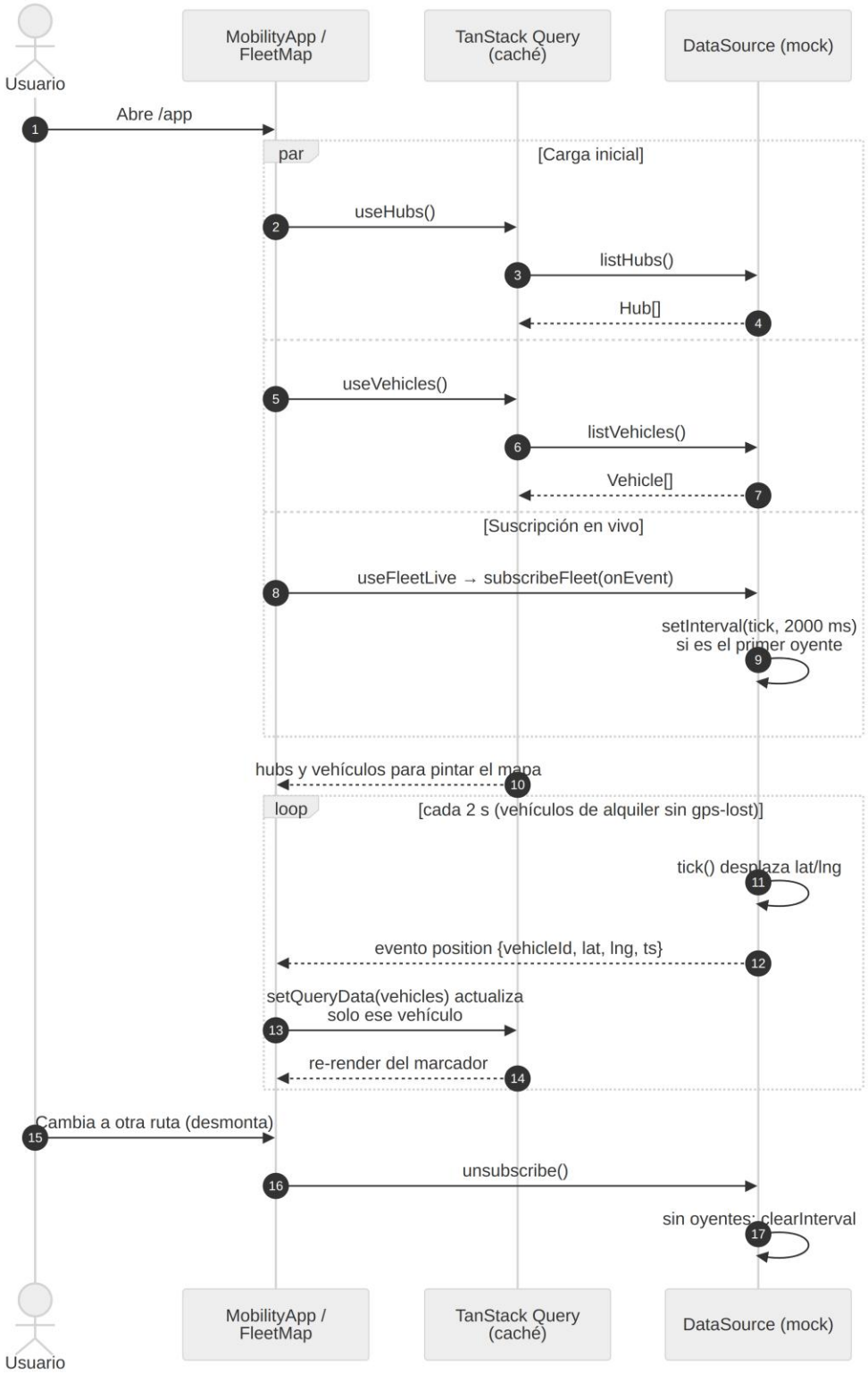
Dos caminos: registro con consentimiento explícito e inicio de sesión. Ambos terminan invalidando la consulta me y redirigiendo con safeRedirect.



2. Mapa en vivo: carga inicial y suscripción a la flota

Requisitos: RF-06, RF-08, RF-17, RNF-01 | **Fuentes:** MobilityApp.tsx, FleetMap.tsx, queries.ts (useFleetLive), mock.ts (subscribeFleet, tick)

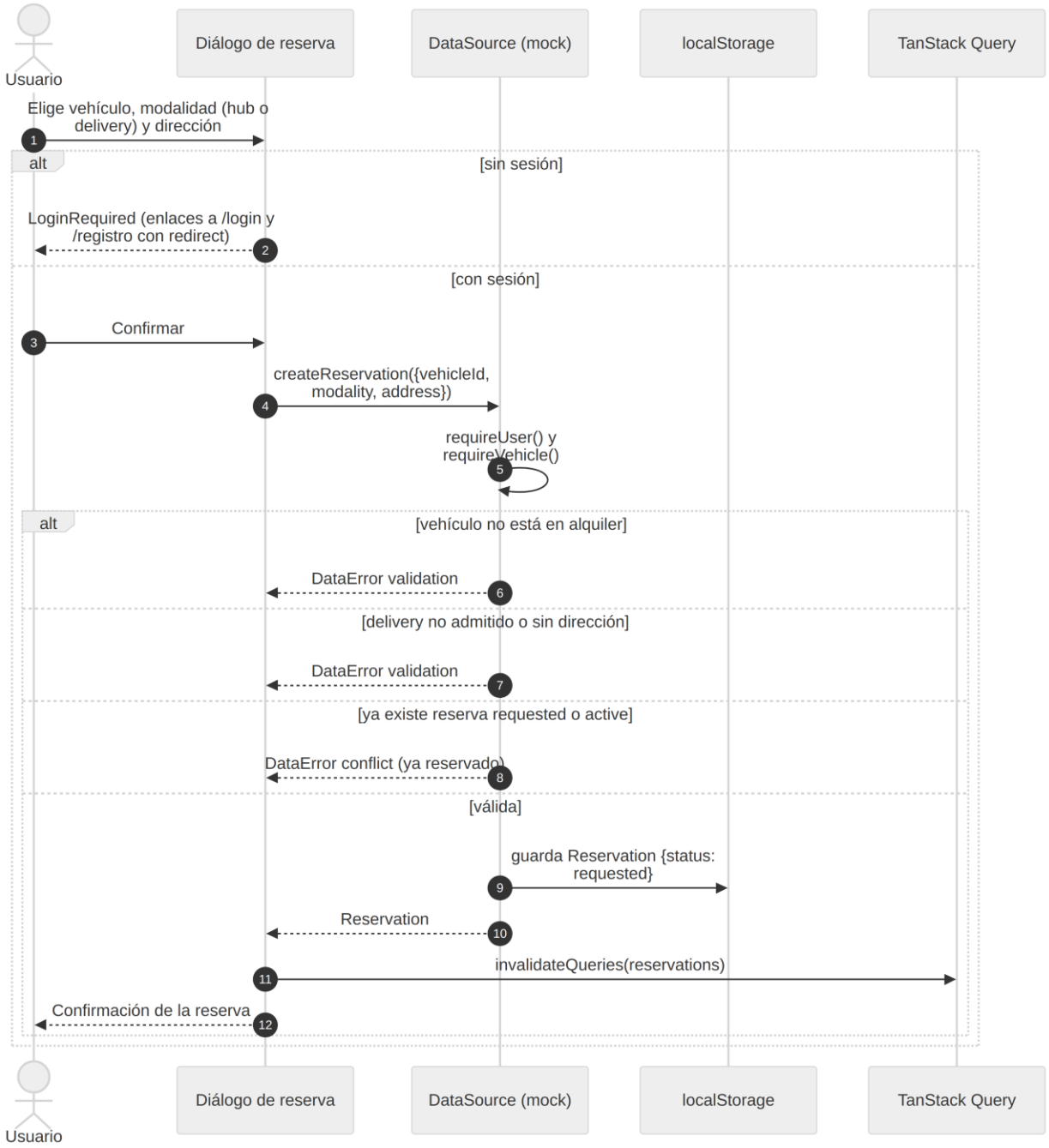
Mapa y panel /admin comparten la misma caché de TanStack Query, por eso leen de una única fuente de datos.



3. Reserva de alquiler en hub o a domicilio

Requisitos: RF-05 | **Fuentes:** CommerceDialogs.tsx (reserva), mock.ts (createReservation), reservations.ts

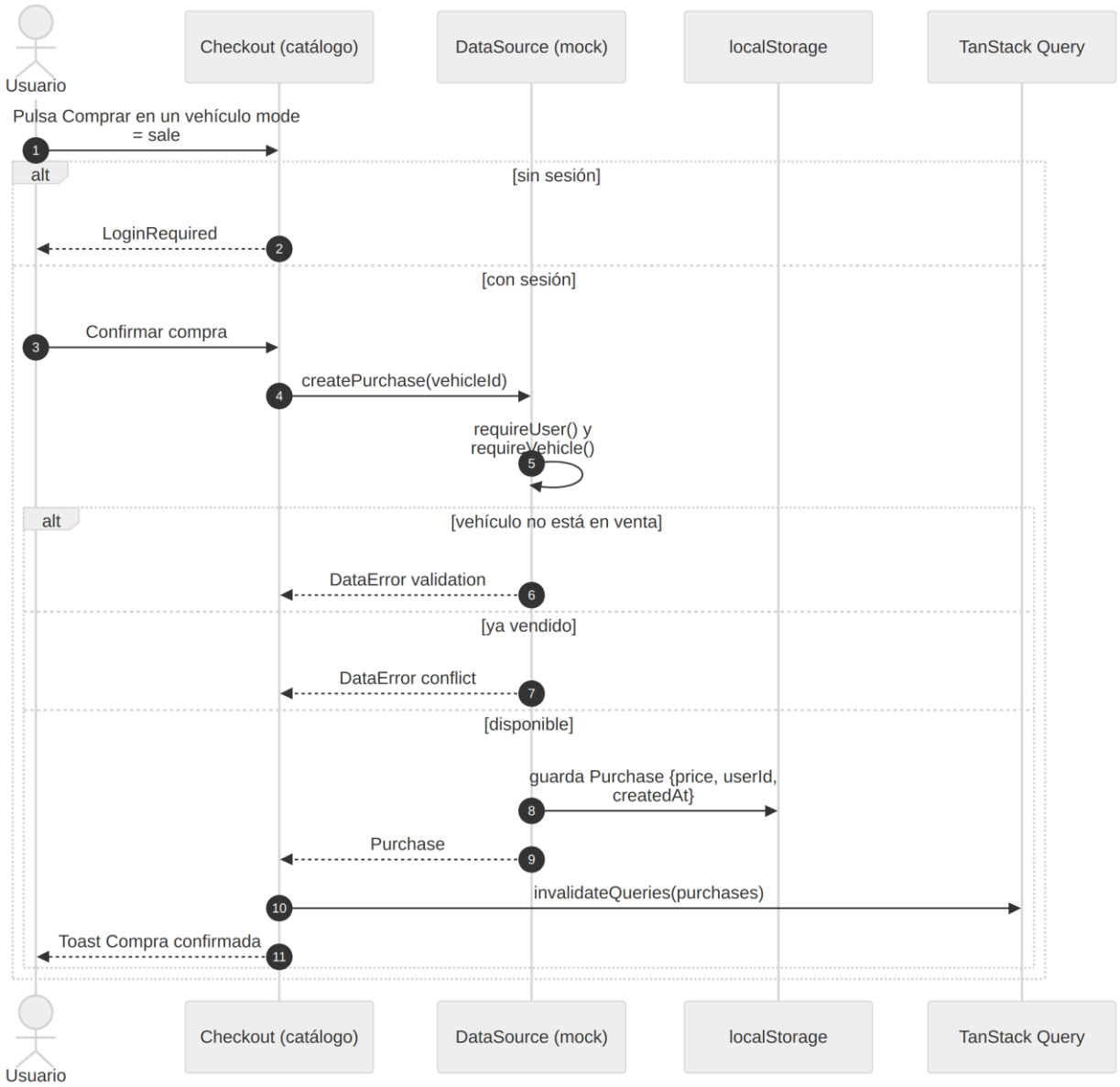
Validaciones: vehículo en alquiler, modalidad delivery admitida, dirección obligatoria y ausencia de reserva activa sobre el mismo vehículo.



4. Compra de un vehículo del catálogo

Requisitos: RF-03 | **Fuentes:** CommerceDialogs.tsx (checkout), mock.ts (createPurchase)

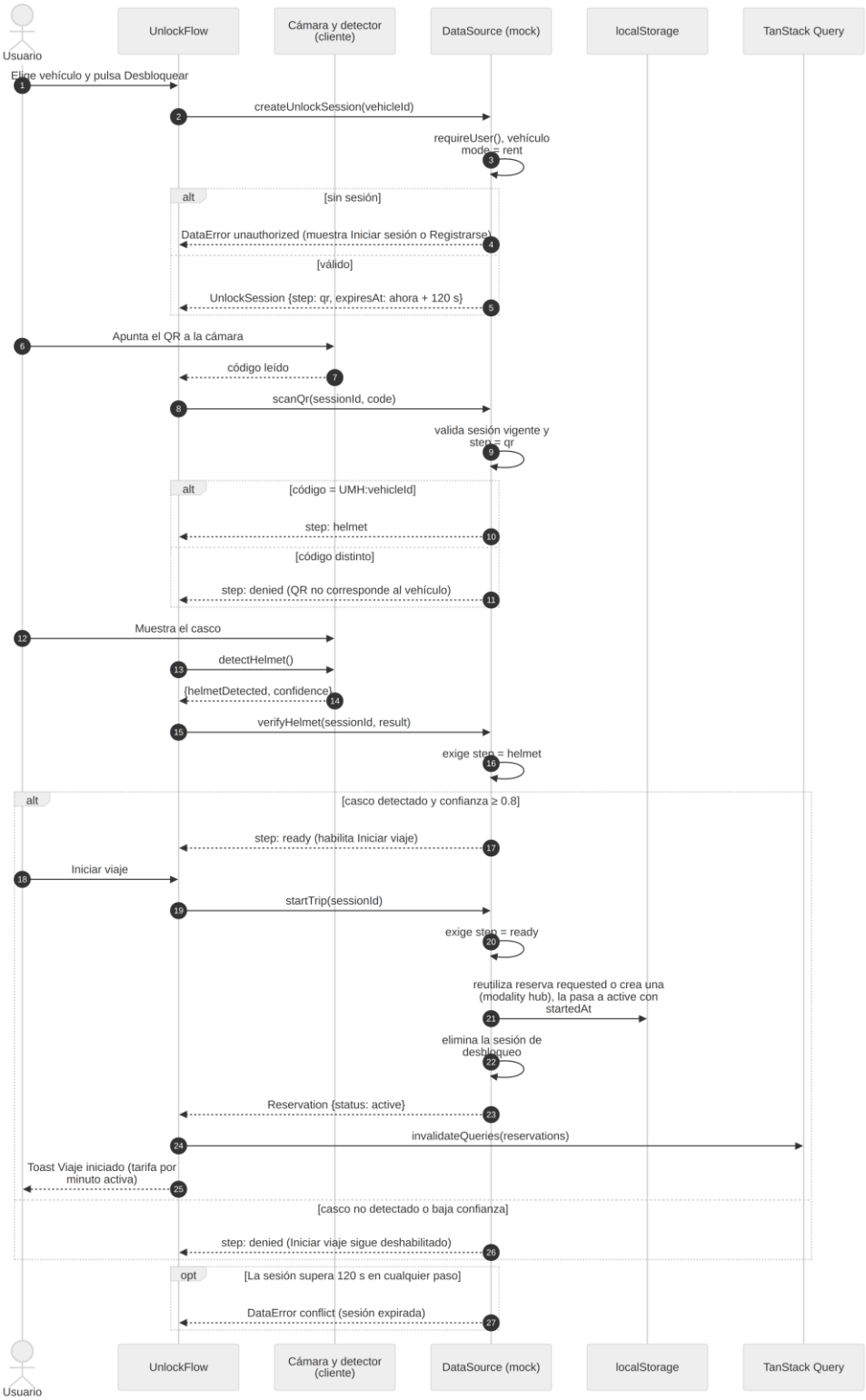
La pasarela de pago es simulada. Un vehículo no puede venderse dos veces.



5. Desbloqueo: QR, casco e inicio de viaje

Requisitos: RF-09 | **Fuentes:** UnlockFlow.tsx, mock.ts (createUnlockSession, scanQr, verifyHelmet, startTrip), server/domain/unlock.ts

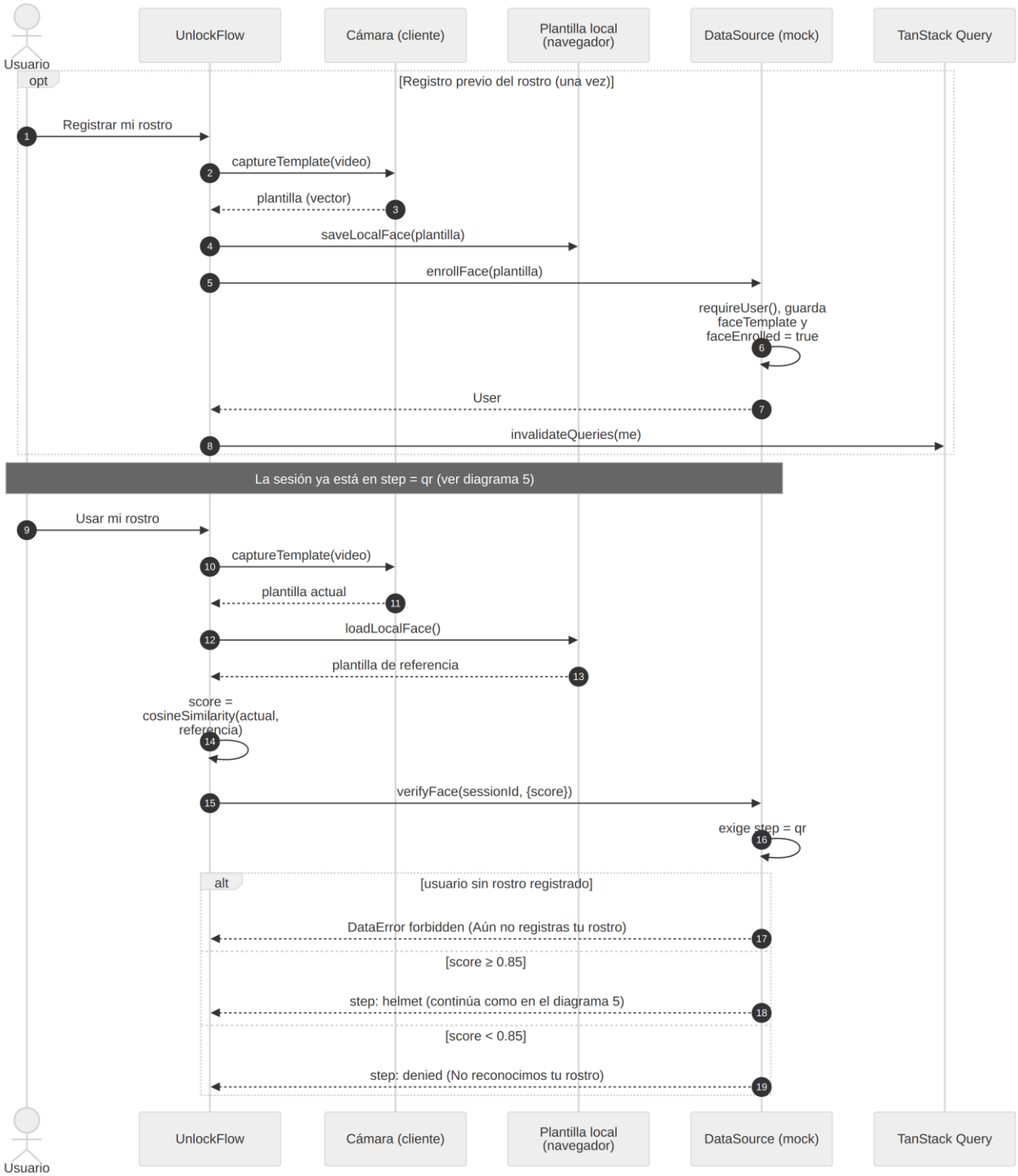
Umbral de casco: confianza ≥ 0.8. TTL de la sesión: 120 s. Código QR válido: UMH:<vehicleId>. En la arquitectura objetivo, startTrip también enviaría un comando UNLOCK por MQTT; el mock no lo hace porque no hay hardware.



6. Desbloqueo alterno por reconocimiento facial y registro del rostro

Requisitos: RF-10, RNF-03 | **Fuentes:** UnlockFlow.tsx (useFace, enrollFace), mock.ts (enrollFace, verifyFace), unlock.ts

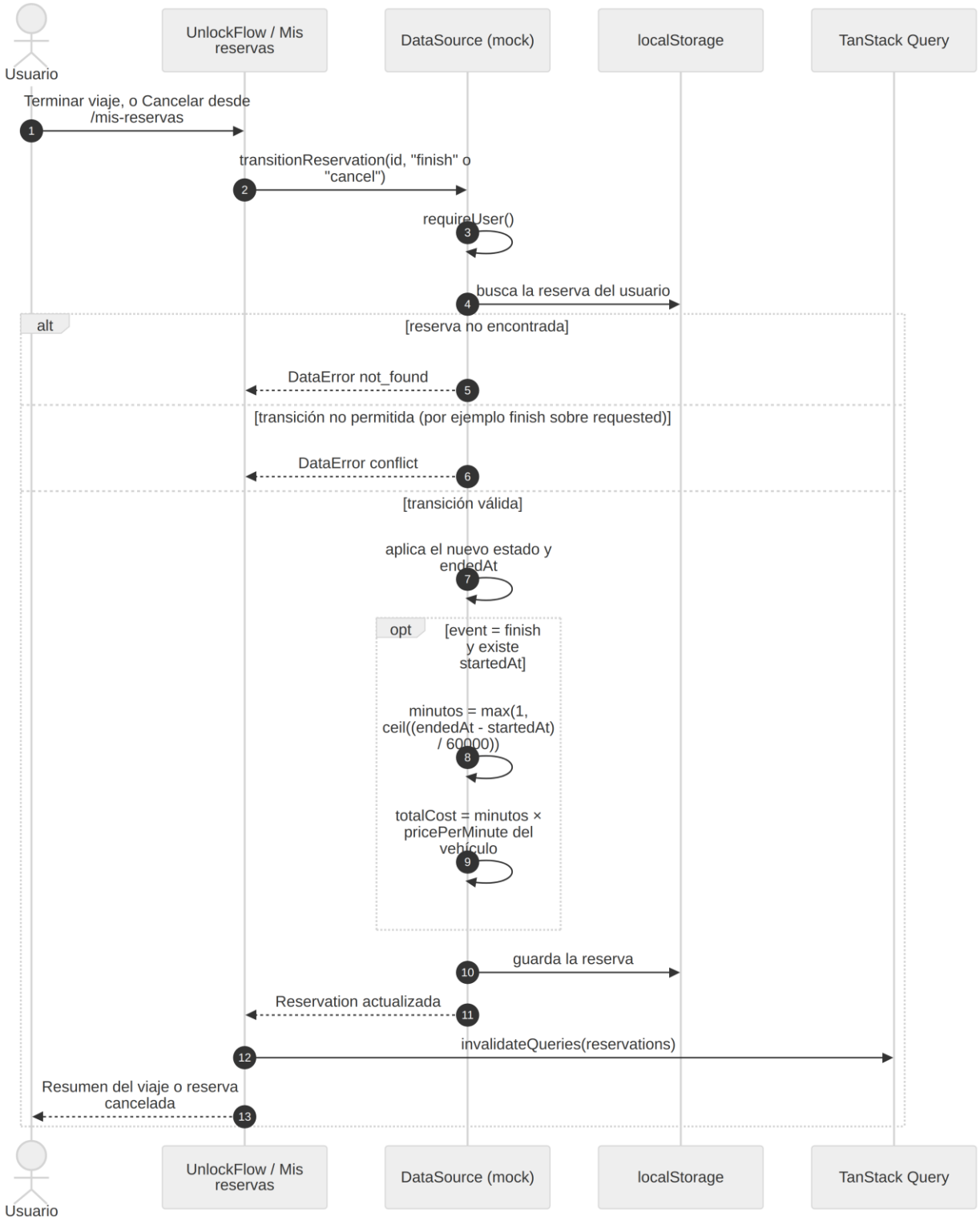
Umbral de similitud: 0.85. La plantilla se extrae en el cliente y se compara por similitud coseno.



7. Gestión de reservas y fin del viaje con cálculo de costo

Requisitos: RF-05 | **Fuentes:** UnlockFlow.tsx (finish), mis-reservas.tsx, mock.ts (transitionReservation), reservations.ts (transition, rentalCost)

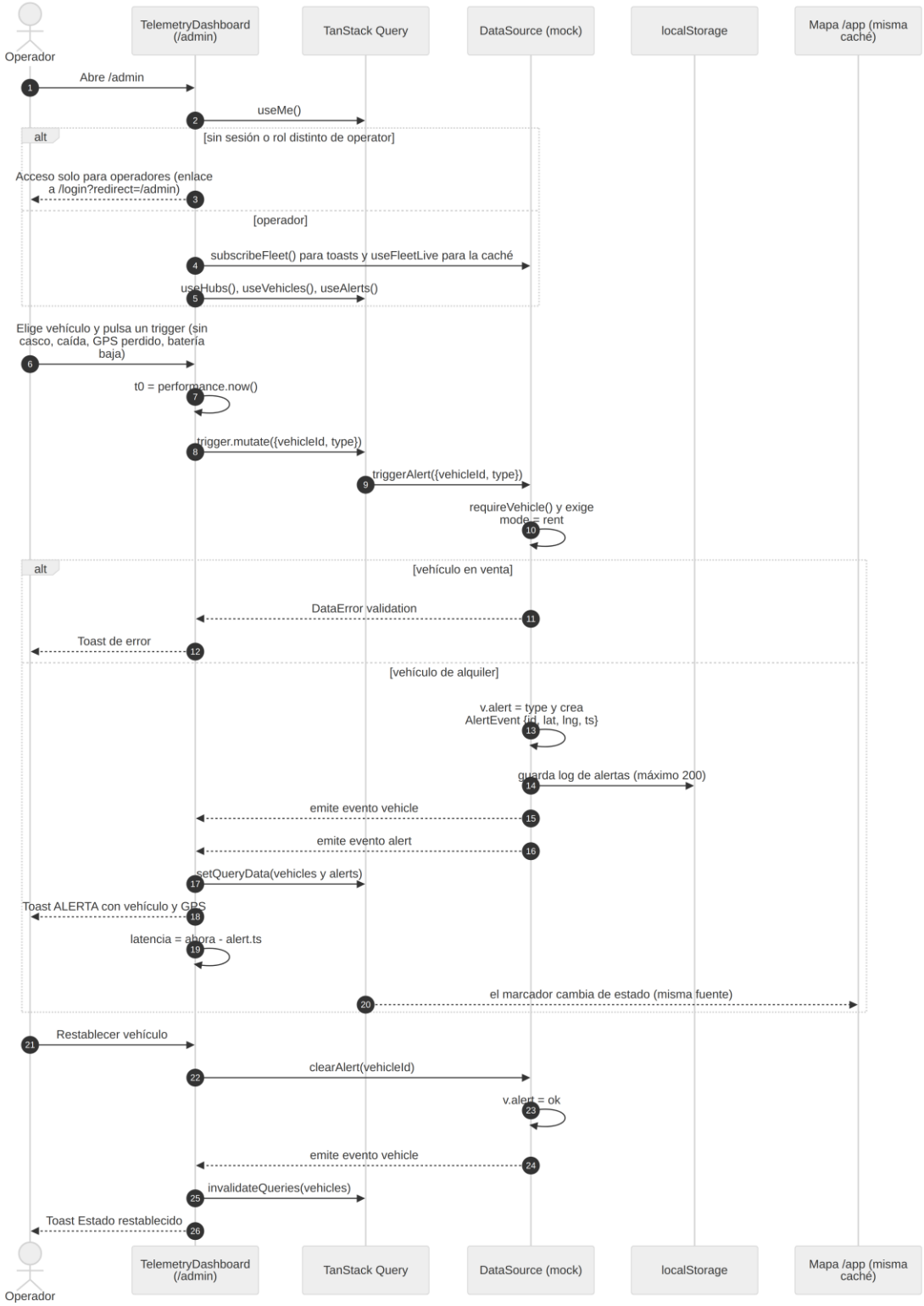
Transiciones válidas: requested → active | cancelled y active → completed | cancelled.



8. Simulador de alertas IoT y resolución por el operador

Requisitos: RF-11, RF-12, RF-15, RNF-02 | **Fuentes:** TelemetryDashboard.tsx, queries.ts (useTriggerAlert, useFleetLive), mock.ts (triggerAlert, clearAlert)

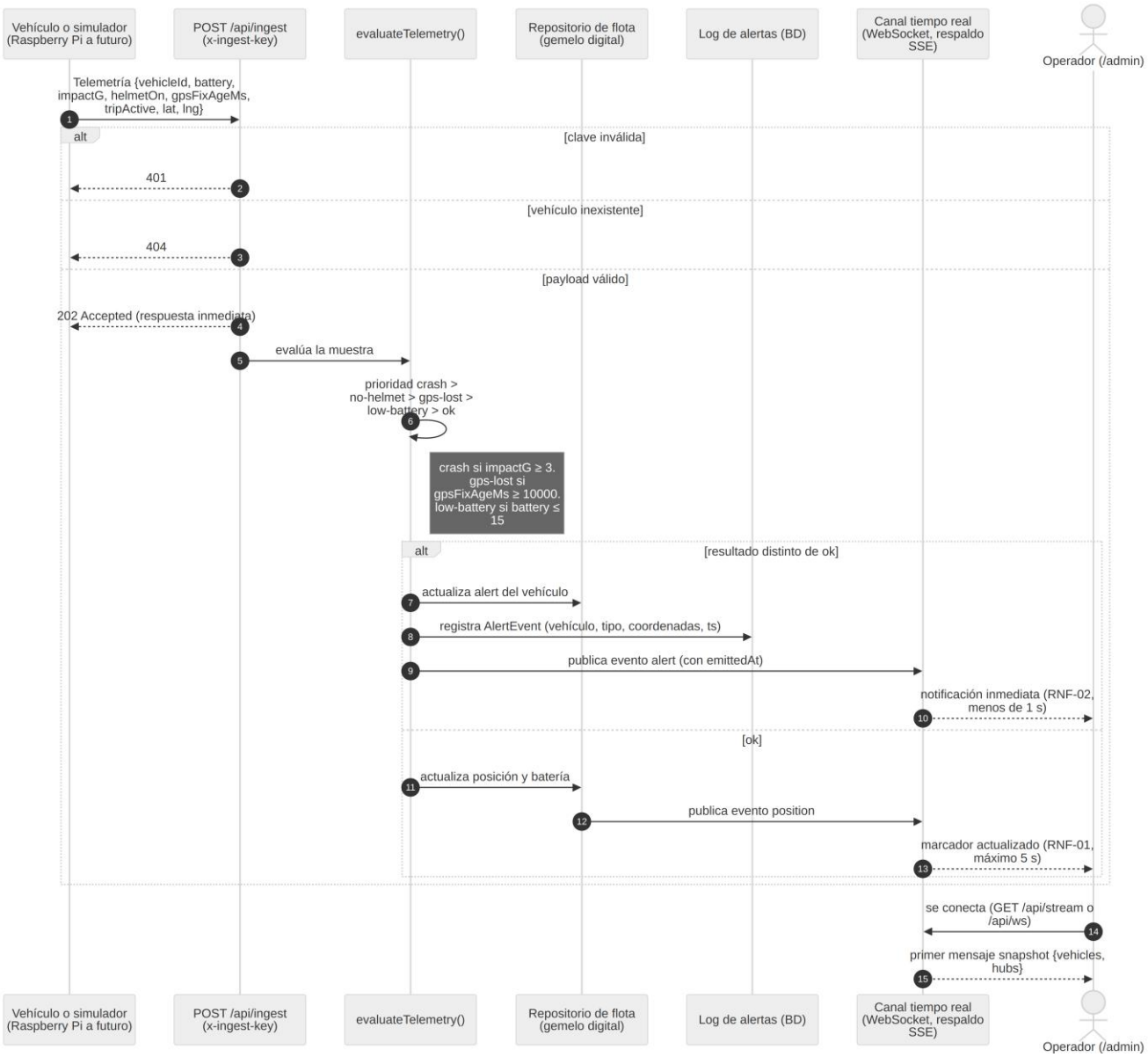
Solo los usuarios con rol operator acceden al panel. El mapa refleja el cambio de estado porque comparte la caché.



9. Telemetría IoT, trigger de seguridad y alerta en tiempo real (arquitectura objetivo)

Requisitos: RF-08, RF-11, RF-12, RNF-01, RNF-02 | **Fuentes:** docs/api/openapi.yaml (/api/ingest, /api/stream, /api/ws), server/domain/alerts.ts, shared/types/realtime.ts

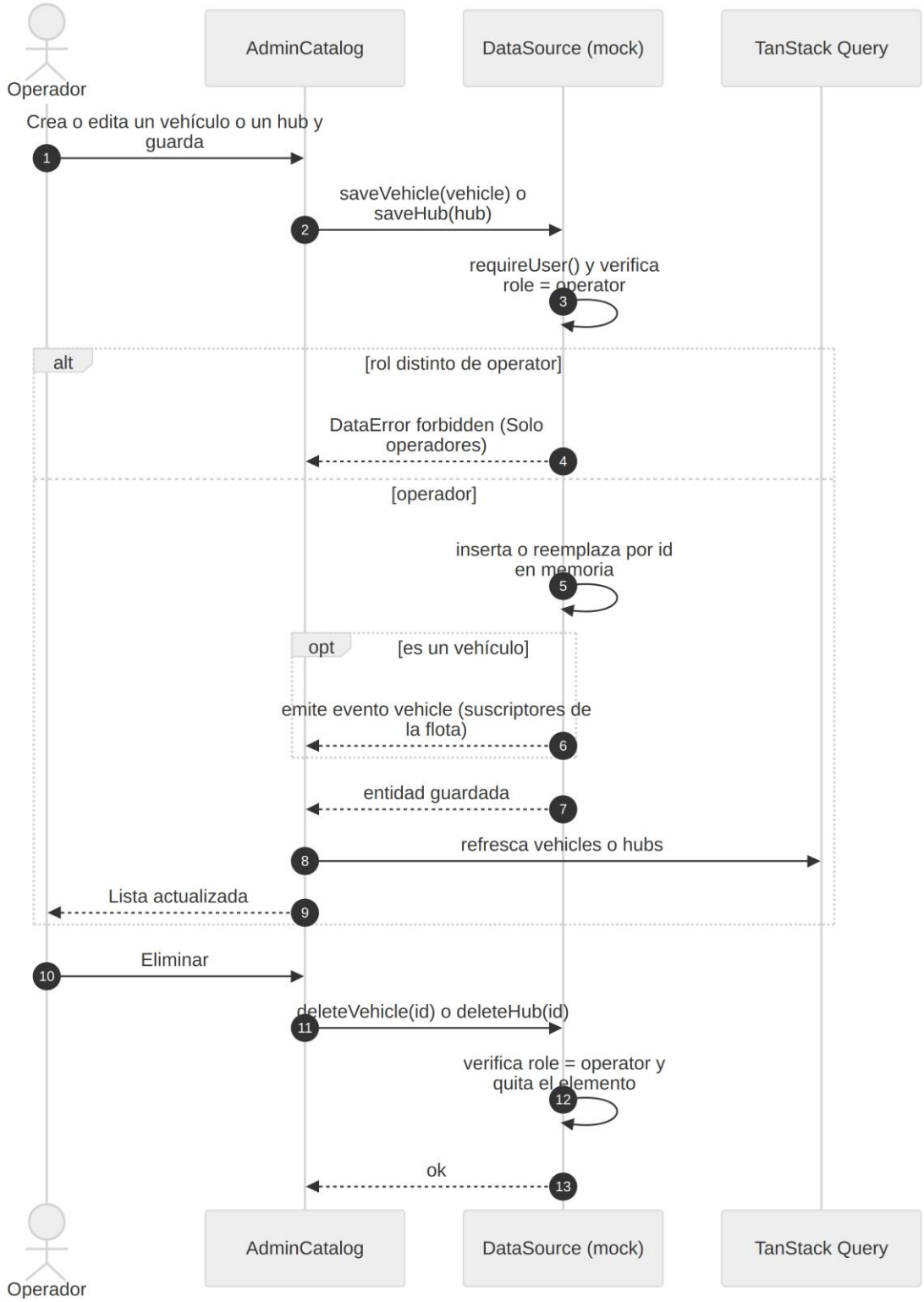
El simulador del operador (/api/admin/simulate-alert) sigue el mismo camino que /api/ingest. Esta ruta aún no está conectada a la UI: hoy el diagrama anterior la reemplaza.



10. Gestión de catálogo y hubs por el operador

Requisitos: RF-21, RNF-08 | **Fuentes:** AdminCatalog.tsx, mock.ts (saveVehicle, deleteVehicle, saveHub, deleteHub)

Todas las operaciones verifican que el usuario tenga rol operator.



Observaciones de coherencia detectadas en el código

1. Reserva previa vs. desbloqueo directo. El OpenAPI exige una reserva requested para abrir la sesión de desbloqueo. El mock, en cambio, crea la sesión sin reserva y startTrip genera una con modalidad hub si no existe. El diagrama 5 sigue el comportamiento del mock.
2. Reglas duplicadas. Los umbrales (0.8 casco, 0.85 rostro, 120 s de TTL) y la máquina de estados viven en src/server/domain/* y también están copiados en mock.ts. Conviene que el mock importe el dominio para evitar divergencias.
3. Verificación facial. En domain/unlock.ts, verifyFace exige registered y consent. El mock solo comprueba faceEnrolled.
4. Costo. rentalCost lanza error con minutos ≤ 0, mientras el mock usa max(1, ceil(...)).
5. MQTT. El diseño en entregables/diseno menciona el comando UNLOCK por MQTT. Ese paso no existe aún en código.